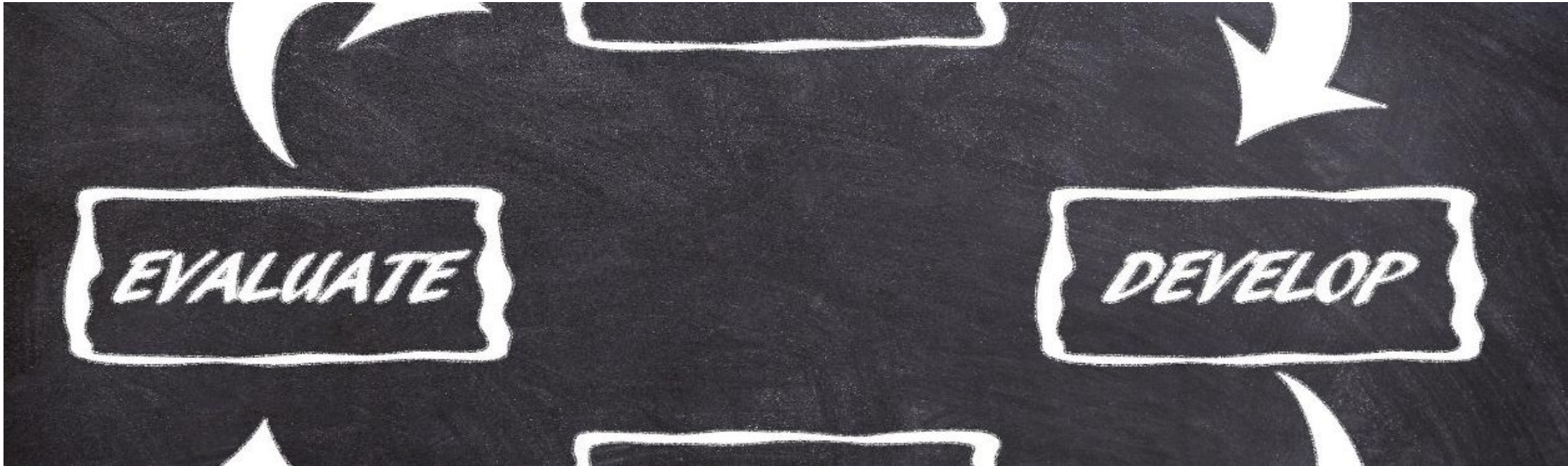




QKI – Qualitätssicherung für KI-Systeme

3. Validierung von maschinell gelernten Modellen

Prof. Dr. Alexander Maier

Inhalte des Moduls

1. Grundlagen von Qualitätssicherung und -management

- I Aufgaben und Ziele von Qualitätsmanagementsystemen im Unternehmen
- I Werkzeuge und Verfahren der Qualitätsplanung, -lenkung, -prüfung und -verbesserung

2. Grundlagen der funktionalen Sicherheit bei technischen Systemen

- I Industriestandards, Zertifizierung
- I Technisch-organisatorische Maßnahmen

3. Validierung von maschinell gelernten Modellen

- I Herausforderungen beim Testen einer KI
- I Strategien und Frameworks zum systematischen Testen

4. Interpretierbare Modelle im maschinellen Lernen (Erklärbare KI, Explainable AI)

- I Informed Machine Learning
- I Strategien und Werkzeuge für Erklärbare KI

5. Zertifizierung einer KI

- I Anforderungen an KI und Zertifizierung
- I Praktische Umsetzung

Inhalte Kapitel 3:

Validierung von maschinell gelernten Modellen

3.1 Datengetriebene und ingenieurmäßige Lösungen

- I Unterschiede und Gemeinsamkeiten

3.2 Unsicherheiten in KI-Systemen

3.3 KI-Testing

- I Herausforderungen beim Testen einer KI
- I Datenqualität
- I Corner Cases

3.4 Assurance Cases als Lösung zur Absicherung von KI?

- I Strategien und Frameworks zum systematischen Testen

Lernziele Kapitel 3: Validierung von maschinell gelernten Modellen

Die Studierenden...

- kennen den Unterschied zwischen datengetriebenen und ingenieurmäßigen Lösungen,
- können anhand der Unsicherheiten beim Einsatz von ML/KI Möglichkeiten zur direkten Steigerung der Qualität benennen,
- kennen die Herausforderungen beim Testen einer KI und sind in der Lage gezielte Maßnahmen zu treffen,
- wenden Strategien und Frameworks zum systematischen Testen einer KI an.

Anmerkung

Wie bereits erwähnt, wird der Begriff „KI“ in dieser Lehrveranstaltung als Black Box betrachtet.
„KI“ beinhaltet alles, was mit Maschinellern Lernen oder einer Datenanalyse in beliebiger Form zu tun hat.

Motivation

I **E-Commerce:**

- I OK, wenn ein Algorithmus in 99% eine passende Empfehlung produziert.

I **Autonomes Fahren:**

- I Inakzeptabel, wenn (nur) in 99% eine richtige Entscheidung getroffen wird.

I **Steigende Erwartungen an die KI**

- I Nur erfüllbar mit ausreichenden Tests
 - I Wie sehen solche Tests aus?
 - I Wer legt die Kriterien fest?
 - I Auf welcher Basis wird getestet?
 - I ...

A3.1.1: Qualität einer KI



Nehmen Sie ihr Szenario aus der Vorbereitungsaufgabe zur Hand.

- I Können in der Anwendung kritische Situationen auftreten, ausgelöst durch die KI? Welche?
- I Was bedeutet „Qualität“ im Kontext Ihrer Anwendung?
- I Wie kann die Qualität sichergestellt werden?
- I Wie sehen die entsprechenden Tests aus?
- I Welche (besonderen) Herausforderungen gibt es dabei?

3.1 Datengetriebene und ingenieurmäßige Lösungen

Intelligente Produkte – aber sicher?

- I Immer mehr Produkte enthalten Komponenten, die auf maschinellen Lernverfahren und künstlicher Intelligenz basieren.
- I einstige Zukunftsvisionen (z.B. autonome Fahrzeuge) rücken in greifbare Nähe
- I Allerdings sind einige **wichtige Fragen zu ML und KI noch nicht abschließend geklärt**
 - I z.B. in welcher Form ein **Nachweis der funktionalen Sicherheit** erbracht werden kann.
- I Nicht nur für autonome Fahrzeuge relevant
- I überall dort wichtig, wo ein **Versagen einer maschinell erlernten Lösung hohe Risiken** mit sich bringt
 - I nicht nur, aber insbesondere für die körperliche Unversehrtheit von Menschen

Intelligente Produkte – aber sicher?

Warum tut sich das klassische Engineering damit schwer?

Für das Verständnis muss folgende Frage geklärt werden:

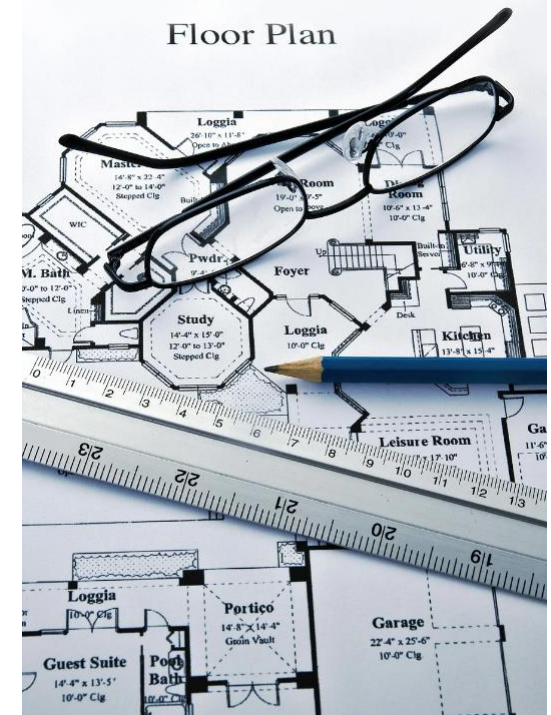
Wie funktioniert das Engineering bis dato und was bewirkt der Einzug von maschinellern Lernen und künstlicher Intelligenz in die zu entwickelnden Produkte?

Ingenieurmäßige Lösungen und klassisches Engineering

Das klassische Engineering wendet etablierte Gesetze und Modelle zur Problemlösung an

Beispiel Bau eines Hochhauses

- I Neben den Bedürfnissen der künftigen Nutzer und der Kreativität des Architekten werden **Modelle zu physikalischen Gegebenheiten** und **Materialeigenschaften** sowie **weitere Regelwerke** berücksichtigt
 - I Bau eines nach dem Stand der Technik, statisch, energetisch und feuerschutztechnisch sicheren Gebäudes.
 - I Aussagen dazu werden im Planungsstadium gemacht und abgenommen.
 - I Die angewandten Gesetzmäßigkeiten, wurden zuvor in zahlreichen empirischen Versuchen weitestgehend als universell gültig validiert
 - I Z.B. hinsichtlich der Verteilung von Traglast und Wärmeausbreitung

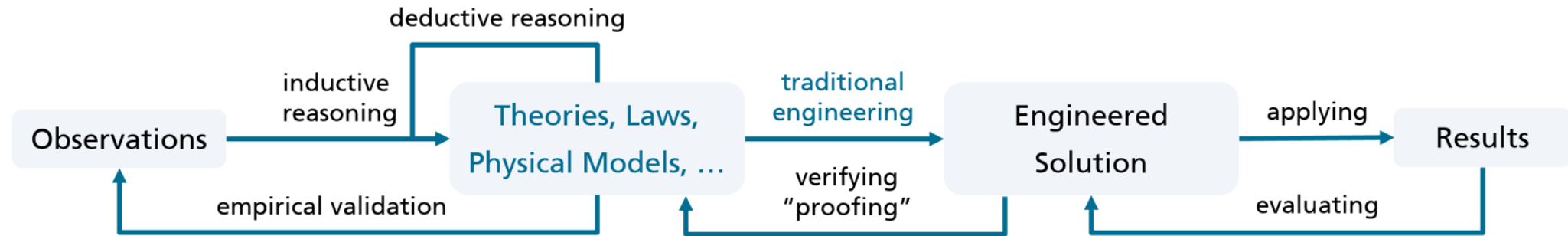


Ingenieurmäßige Lösungen und klassisches Engineering

Bei der ingenieurmäßigen Problemlösung liegt der Fokus auf einer **geschickten und korrekten Ausnutzung dieser Gesetzmäßigkeiten**, um die gestellten Anforderungen zu erfüllen.

- I So lässt sich eine Lösung häufig nach ihrer initialen Konzeption auf **Basis der zugrundeliegenden Gesetzmäßigkeiten verifizieren**, beispielsweise, ob die zugrundeliegende Baustatik die benötigte Tragfähigkeit bietet.

Ingenieurmäßige Lösungen und klassisches Engineering



Ingenieurmäßige Lösungen und klassisches Engineering

- I Verallgemeinerung:
 - I der frühzeitige konzeptionelle Nachweis bestimmter Eigenschaften ist ein wichtiges Merkmal einer ingenieurmäßigen Lösung
- I Dies gilt nicht durchgängig für alle Eigenschaften einer Lösung
 - I Umsetzung ist in den meisten Disziplinen von zusätzlichen Evaluationsaktivitäten begleitet
 - I Bei Bauwerken sind dies Begehungen und Abnahmen
 - I bei Automobilen werden Simulationen und Testfahrten durchgeführt

A3.1.2: Ingenieurmäßige Lösungen und klassisches Engineering



Warum ist diese ingenieurmäßige Herangehensweise aus dem klassischen Engineering für den Einsatz von ML/KI problematisch?

Ingenieurmäßige Lösungen und klassisches Engineering

- I Auch das **Software Engineering** folgt weitgehend der Tradition der ingenieurmäßigen Konzeption von Lösungen
 - I vergleichbar dem Systems Engineering
- I Ein Problem wird konkretisiert, in Teilprobleme zerlegt und die sukzessiv verfeinerte Lösung letztlich mittels geeigneter Algorithmen durch die Entwickler (Software-Ingenieure) im Programmcode abgebildet.
 - I z.B. V-Modell, Wasserfall, etc.

Was unterscheidet datengetriebene von ingenieurmäßigen Lösungen?

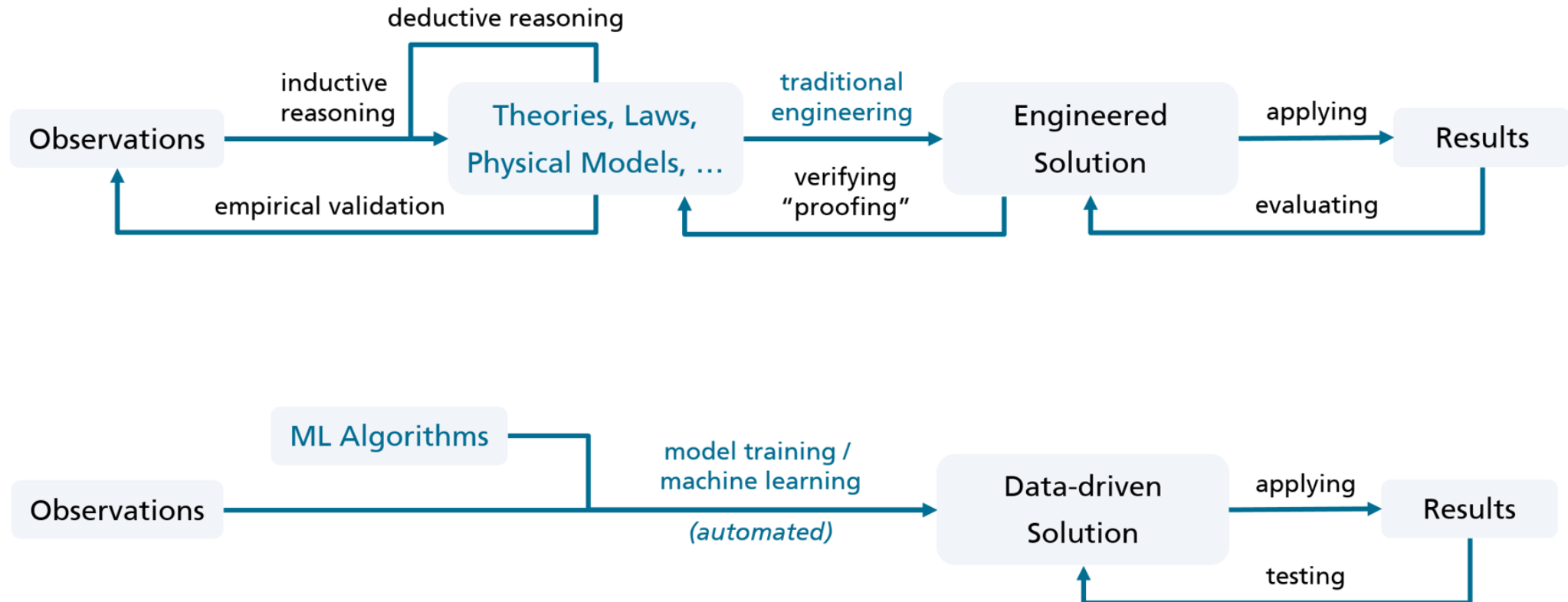
- I **Klassisches Engineering:** Mensch kombiniert geschickt Gesetzmäßigkeiten zur Problemlösung und basiert auf Erfahrung
- I **Datengetriebene Problemlösungen** weisen in ihrer Entstehung einen **fundamentalen Unterschied** zu ingenieurmäßigen Lösungen auf.
- I der maschinelle Lernalgorithmus generalisiert aus den bereitgestellten Beobachtungsdaten ein datenbasiertes Modell als spezifische Lösung für die konkrete Problemstellung
- I Beispiele für solche Problemstellungen
 - I Erkennen von Verkehrsschildern im Straßenverkehr
 - I Identifikation von Tumoren auf CT-Bildern

Was unterscheidet datengetriebene von ingenieurmäßigen Lösungen?

Zugrundeliegende Gesetzmäßigkeiten und Zusammenhänge brauchen bei dieser Art der Problemlösung dem Ingenieur im Vorfeld **nicht bekannt** zu sein

→ Datengetriebene Lösungen nehmen eine Abkürzung, indem sie den wissenschaftlichen Schritt der Theoriebildung und experimentellen Validierung überspringen.

Was unterscheidet datengetriebene von ingenieurmäßigen Lösungen?



Was unterscheidet datengetriebene von ingenieurmäßigen Lösungen?

- I Datengetriebene Lösungen besitzen gegenüber ingenieurmäßigen Lösungen sowohl Vorteile als auch Nachteile.
- I mittels **maschinellen Lernverfahren** lassen sich insbesondere auch Lösungen für Probleme bereitstellen, die entweder derzeit ingenieurmäßig **nicht sinnvoll lösbar** sind oder deren aktuelle ingenieurmäßige Lösungen bezüglich **Effektivität und Effizienz bei weitem übertroffen** werden
 - I Aktuelle Beispiele sind die Identifikation von Objekten in Bilddaten, die Erkennung menschlicher Sprache wie auch die automatische Übersetzung von Texten

Was unterscheidet datengetriebene von ingenieurmäßigen Lösungen?

Mögliche Lösung:

- I Gleichsetzung der angewandten maschinellen Lernverfahren mit den Gesetzmäßigkeiten im traditionellen Engineering**
 - I Immerhin werden die Methoden des ML ebenfalls durch den Ingenieur geeignet ausgewählt und zur Lösung eines Problems angewendet.
 - I Den potenziellen Nachteilen einer datengetriebenen Lösung könnte dementsprechend durch ein ingenieurmäßiges Vorgehen bei Erstellung und Einsatz der Algorithmen entgegnet werden.

A3.1.3: Gleichsetzung der angewandten maschinellen Lernverfahren mit den Gesetzmäßigkeiten im traditionellen Engineering



Warum können die Methoden der angewandten maschinellen Lernverfahren nicht mit den Gesetzmäßigkeiten im traditionellen Engineering gleichgesetzt werden?

Was unterscheidet datengetriebene von ingenieurmäßigen Lösungen?

- I Die Algorithmen der maschinellen Lernverfahren selbst stellen keine Gesetzmäßigkeiten dar
 - I Dadurch können sie nicht grundsätzlich empirisch validiert werden
- I Zudem ist bei datengetriebenen Lösungen **nicht der Ingenieur derjenige, der die Lösung konzipiert**
- I Der Ingenieur gibt lediglich einen oder mehrere **Algorithmen** vor
- I Die **Algorithmen erlernen selbstständig** anhand vorhandener Daten eine Lösung
 - I Abstrahieren **aus den Daten** ein passendes Modell

Wie wichtig ist uns das „Warum“ bei datengetriebenen Lösungen?

- I Das Vorhandensein einer **Begründung** ist für Menschen ein **wichtiges Akzeptanzkriterium**.
 - I Vergleich: Die Urteilsbegründung in Gerichtsverfahren ist gewöhnlich deutlich aufwändiger als die reine Urteilsfindung.
- I Beim **Einsatz aktueller maschineller Lernverfahren** ist es im Allgemeinen jedoch **schwierig**, ein bestimmtes Ergebnis anhand der Berechnungen im zuvor erlernten Modell für einen Menschen **nachvollziehbar zu begründen**.

Wie wichtig ist uns das „Warum“ bei datengetriebenen Lösungen?

- I Aktuelle Forschungsaktivitäten beschäftigen sich mit der Verbesserung dieser Situation
- I Paradox:
 - I Sobald die Ergebnisse einer maschinell erlernten Lösung vollständig mittels Begründungen nachvollziehbar sind, ist das Problem auch ingenieurmäßig modellierbar und damit lösbar.
- I Solange diese **Tiefe an Verständnis nicht erreicht** ist, handelt es sich hingegen im besten Fall um eine **Plausibilisierung der Ergebnisse**.

Wie wichtig ist uns das „Warum“ bei datengetriebenen Lösungen?

Wenn wir von aktuellen Fortschritten im Bereich künstlicher Intelligenz und des maschinellen Lernens profitieren wollen, **müssen wir voraussichtlich damit leben**, dass wir die **Frage nach dem „warum“** hinter der Lösung **nicht immer befriedigend beantworten können**.

Was auch **bei datengetriebenen Lösungen erstrebenswert** bleibt, ist eine möglichst **verlässliche Einschätzung der Unsicherheit** im bereitgestellten Ergebnis

I (natürlich neben dem Ausschluss erkennbarer Fehlerquellen)

A3.1.4: Wie wichtig ist uns das „Warum“ bei datengetriebenen Lösungen?



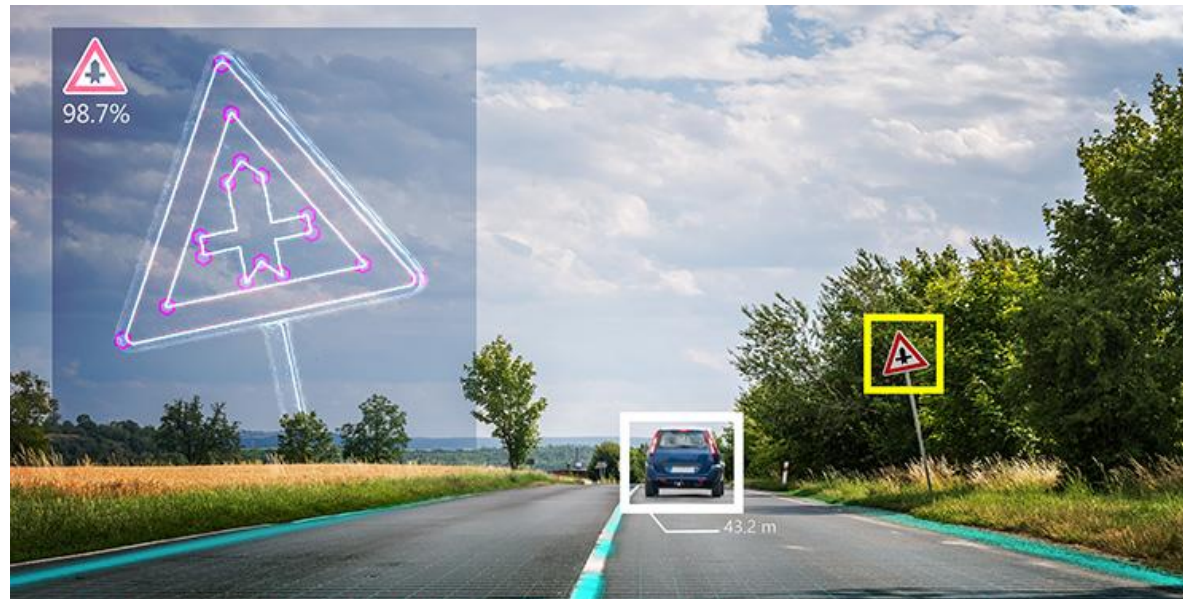
- Können KI/ML-Verfahren aus Sicht der Funktionalen Sicherheit und der Zertifizierung eingesetzt werden, auch wenn die Frage nach dem „Warum“ nicht (endgültig) geklärt ist?
- Warum? Warum nicht?

3.2 Unsicherheit in KI-Systemen

Wissen um die Unsicherheit

Beispiel:

- Eine datengetriebene Komponente zur Verkehrsschilderkennung
 - sollte nicht nur die Information liefern, dass ihre Auswertung auf das Vorhandensein eines Vorfahrtsschildes schließen lässt
 - sondern auch Information zur im Ergebnis enthaltenen Unsicherheit und seiner Konfidenz in deren Bestimmung.



Datengetriebene Lösung zur
Verkehrsschildbestimmung

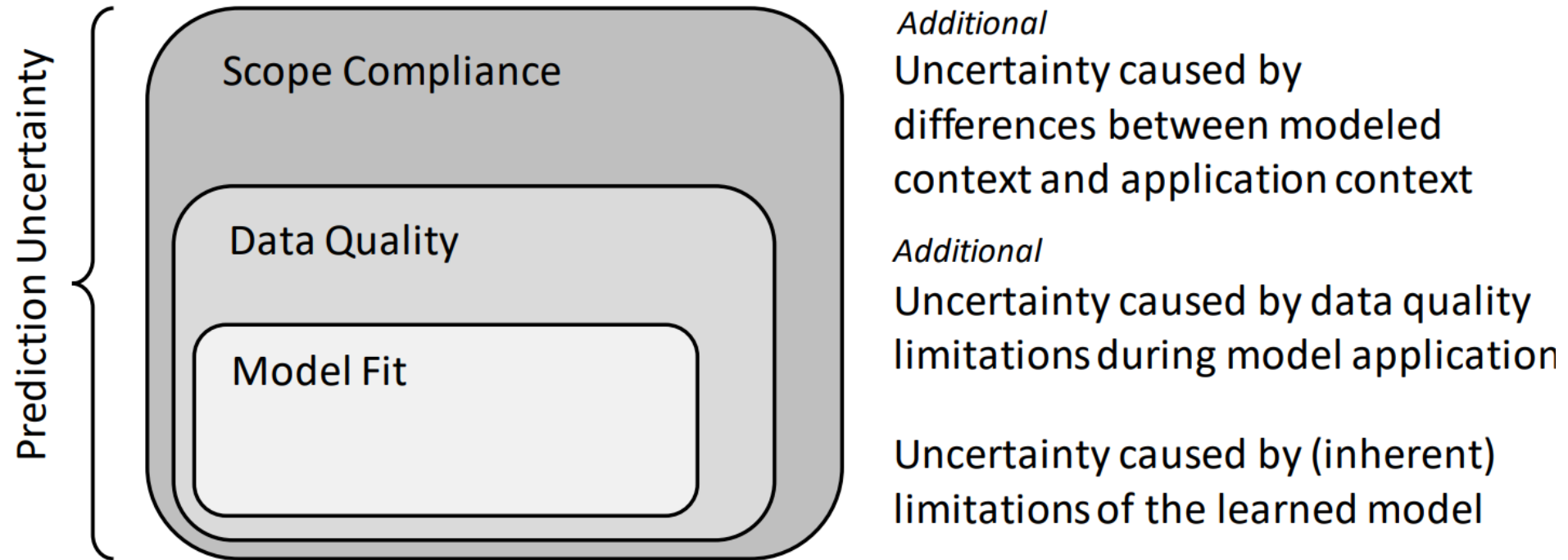
Wissen um die Unsicherheit

- I Das könnte im Beispiel wie folgt aussehen
 - I „unter Berücksichtigung der aktuellen Licht- und Wetterverhältnisse sowie dem Verschmutzungsgrad der Frontkamera wurde das Schild auf einem Konfidenzniveau von 0.95 mit einer Wahrscheinlichkeit von > 98,6 % als Vorfahrtsschild erkannt“.
- I Bei der **Unsicherheit** im Ergebnis spielen hierbei neben der generellen Qualität des Modells auch der **aktuelle Einsatzkontext** wie auch die **Qualität der zur Verfügung stehenden Daten** eine wichtige Rolle.
- I Es ist leicht nachzuvollziehen, dass beispielsweise das Wissen um **schlechte Lichtverhältnisse** oder eine **verschmutzte Kameralinse** wenig Informationsgewinn bei der korrekten Bestimmung eines Verkehrsschildes liefern.
 - I Sie helfen jedoch, die Unsicherheit im bereitgestellten Ergebnis akkurater zu beurteilen und damit dem diese Information verwendenden Gesamtsystem bessere Entscheidungen zu treffen.

Unsicherheit in ML/KI

- I Vorhersageunsicherheit ist die Wahrscheinlichkeit, dass das gelieferte Ergebnis eines Modells falsch sein oder einen bestimmten Genauigkeitsbereich überschreiten kann
 - I basierend auf dem Modellbildungs- und Testprozess
- I **drei Hauptquellen der Unsicherheit** [Klās 2018]:
 - I Modellanpassung (Model fit)
 - I Datenqualität
 - I Scope-Compliance
- I die drei Unsicherheitskategorien sind übereinander gestapelt
→ Schichtmodell (nächste Folie)

Unsicherheit in ML/KI



Unsicherheit in ML/KI

1. Model Fit

- I Unsicherheit in Bezug auf die Modellanpassung (Model Fit)
 - I wird dadurch verursacht, dass **KI/ML-Techniken empirische Modelle** liefern, die nur eine **Annäherung** an die reale (funktionale) Beziehung zwischen der Modelleingabe und ihrem Ergebnis darstellen.
- I Die Genauigkeit dieser Näherung repräsentiert die Modellanpassung
 - I z.B. aufgrund der begrenzten Anzahl von Modellparametern, der berücksichtigten Eingangsvariablen und zum Trainieren des Modells verfügbaren Daten

Unsicherheit in ML/KI

1. Model Fit

Es gibt zwei wichtige zugrunde liegende Annahmen bezüglich der durch die Modellanpassung verursachten Unsicherheit.

1. Das Modell wird in einer **Umgebung** angewendet, die durch den **Testdatensatz angemessen repräsentiert** wird
 - ┃ Gilt für den bereinigten Datensatz, aus dem die Testdaten typischerweise zufällig ausgewählt werden
2. Das Modell wird auf **Daten** auf einem homogen **hohen Qualitätsniveau** aufgebaut und angewendet
 - ┃ d.h. die Daten leiden nicht unter Qualitätsproblemen
 - ┃ was auf einen gut bereinigten Datensatz zutreffen sollte

Unsicherheit in ML/KI

1. Model Fit

Implikation:

- I Die durchschnittliche **Unsicherheit**, die durch die **Modellanpassung** verursacht wird
 - I kann mit Standardmodellbewertungsansätzen gemessen werden, die auf qualitativ hochwertige Daten angewendet werden,
 - I kann als Annäherung an die untere Grenze der verbleibenden Unsicherheit angesehen werden.

Unsicherheit in ML/KI

2. Datenqualität

- I In einer **realen Umgebung** sind alle Arten von gesammelten Daten
 - I in ihrer **Genauigkeit** begrenzt und
 - I möglicherweise durch verschiedene **Arten von Qualitätsproblemen** beeinflusst.
 - I z.B. basierend auf Sensoren, aber auch menschlichen Eingaben
- I Die tatsächliche Unsicherheit des Ergebnisses eines **KI/ML-Modells wird somit von der Qualität** (insbesondere der Genauigkeit) **der Daten beeinflusst**, auf die es derzeit angewendet wird.

Unsicherheit in ML/KI

2. Datenqualität

- I Daher kann die **zusätzliche Unsicherheit**, die sich aus einem Delta zwischen der Qualität der bereinigten Daten und den Daten ergibt, auf die das Modell aktuell angewendet wird, als Datenqualitäts-(verursachte) Unsicherheit definiert werden.
 - I **Unterschied zwischen Labor und „echtem Leben“**
- I In unserem Beispiel kann das Vertrauen in das Modellergebnis durch eine Kamera mit geringerer Auflösung oder ein beschädigtes Objektiv sowie durch schwierige Beleuchtung und schlechte Wetterbedingungen wie Regen oder Nebel beeinträchtigt werden.

Unsicherheit in ML/KI

2. Datenqualität

Implikation:

- I Der Umgang mit Datenqualitätsunsicherheiten erfordert die **Erweiterung der Standardmodellbewertungsverfahren** um spezielle Setups, um die **Auswirkungen verschiedener Qualitätsprobleme** auf die Genauigkeit der Ergebnisse zu untersuchen
- I **Unsicherheitsanpassungen** für Fälle bereitstellen, in denen das Modell auf **Daten unter dem geforderten Qualitätsniveau** angewendet wird
- I Infolgedessen wurde die Datenqualität quantifiziert und gemessen, um Rohdaten nicht nur während der Datenaufbereitung mit **Qualitätsinformationen** zu versehen, sondern auch um die aktuelle Qualität der Daten **nach der Bereitstellung des Modells** zu messen.

Unsicherheit in ML/KI

3. Scope Compliance (Einhaltung des Geltungsbereichs)

- I KI/ML-Modelle werden für **einen bestimmten Kontext** entwickelt und in diesem getestet.
- I Werden diese Modelle **außerhalb dieses Kontextes** angewendet, können ihre **Ergebnisse unzuverlässig** werden
 - I z.B. weil das Modell extrapolieren muss
- I Daher kann die **Wahrscheinlichkeit, dass ein Modell derzeit außerhalb des Geltungsbereichs**, für den es getestet wurde, angewendet werden, um die (verursachte) **Unsicherheit bezüglich der Einhaltung des Geltungsbereichs** zu beschreiben.

Unsicherheit in ML/KI

3. Scope Compliance (Einhaltung des Geltungsbereichs)

- I In unserem Beispiel würde das Vertrauen in das Ergebnis des Modells **stark beeinträchtigt**, wenn das an **deutschen Verkehrszeichen trainierte** und getestete Modell in einem **anderen Land** angewendet würde, das sich nicht an die Wiener Verkehrszeichenkonvention hält.
- I Wenn die Rohdaten für die Modellentwicklung und den Test zwischen Mai und Oktober erhoben würden, würden im Testdatensatz zudem höchstwahrscheinlich Bilder von (teilweise) schneebedeckten Verkehrszeichen fehlen.

Unsicherheit in ML/KI

3. Scope Compliance (Einhaltung des Geltungsbereichs)

Implikation:

Die Unsicherheit bei der Einhaltung des Geltungsbereichs kann aus zwei Quellen resultieren, wie im Beispiel veranschaulicht:

1. Das **Modell kann außerhalb des beabsichtigten Geltungsbereichs** angewendet werden
 - ┃ kann erkannt werden, indem relevante Kontextmerkmale überwacht und die Ergebnisse mit den Grenzen des beabsichtigten Umfangs verglichen werden.
 - ┃ (in unserem Beispiel, z.B. GPS-Standort, Geschwindigkeit, Temperatur, Datum, Uhrzeit)
2. Der **Rohdatensatz ist möglicherweise nicht repräsentativ** für den beabsichtigten Geltungsbereich.
 - ┃ Um letzteres zu begründen, müssen Rohdaten mit Kontextmerkmalen annotiert werden, um ihre tatsächliche und angenommene Verteilung im beabsichtigten Umfang zu vergleichen.
 - ┃ (z.B. GPS-Standort, Geschwindigkeit, Temperatur, Datum, Uhrzeit)

A3.2.1: Unsicherheit in ML/KI



Nehmen Sie nochmal ihr Szenario aus der Vorbereitungsaufgabe zur Hand.

- I Untersuchen Sie Ihre Anwendung bzgl. der drei Dimensionen der Unsicherheit nach [Kläs 2018].
- I Sollte Ihr Szenario unter den Dimensionen „nicht viel hergeben“, wählen Sie ein anderes Szenario.
 - I Weitere Beispiele:
 - I KI für Autonomes Fahren
 - I KI für Qualitätsüberwachung der Produkte einer Produktion

A3.2.2: Unsicherheit in ML/KI



- I Mit welchen Maßnahmen lässt sich die Qualität einer KI insbesondere unter Berücksichtigung der Dimensionen der Unsicherheit nach [Klås 2018] steigern?

3.3 KI-Testing

Qualität von KI-Systemen

- I Bei klassischer Software ist das **Testen ein wichtiger** und in der Informatik **etablierter Bereich**, der viele verschiedene Methoden, Ansätze und best practices zur Überprüfung von Software bietet.
- I Oftmals ist bereits die **Entwicklung auf das Testen ausgerichtet** und es gibt sogar das Berufsbild der Softwaretester.



- I Im Vergleich steckt bei **KI das Thema Qualitätssicherung** noch in den **Kinderschuhen** und angesichts der Eigenschaften von Maschinellern lassen sich bekannte Verfahren und Methoden auch nur eingeschränkt übertragen.

Qualität von KI-Systemen

- I Qualität von KI-Systemen ist von entscheidender Bedeutung,
 - I Versagen kann weitreichende Konsequenzen mit sich bringen
 - I z.B. Autonomes Fahren
 - I KI lernt auf Basis von Daten
 - I Es gibt keine klar vorgegebenen Algorithmen
 - I Ziele können unterschiedlich sein
 - I Entscheidungen beruhen auf unterschiedlichen Grundlagen
 - I Algorithmus
 - I Daten
- der Test von KI-Systemen benötigt ganz neue Herangehensweisen

Qualität von KI-Systemen

Herkömmliche Software (klassische softwarebasierte Systeme):

- I Funktionalität und Qualität hängen in erster Linie vom geschriebenen und für Entwickler generell nachvollziehbaren Softwarecode ab
- I Grundsätzlich sollte es möglich sein, alle kritischen Fehler zu finden und zu beseitigen
- I **Funktionalität wird durch die Entwickler über Befehle vorgegeben**, die eine bestimmte Eingabe eindeutig in eine Ausgabe überführen
- I Testen konzentriert sich dann darauf, die Zuverlässigkeit des Systems zu untersuchen, indem anhand von Testfällen geprüft wird, ob die spezifizierten Befehle Eingaben in die erwarteten Ausgaben überführen.

Testen in klassischen softwarebasierten Systemen

Datengetriebene Komponenten:

- I Funktionalität wird nicht durch Befehle vorgegeben, sondern durch ein (hochgradig) parametrisiertes Modell bereitgestellt, dessen Parameter mittels eines vorgegebenen Lernverfahrens **automatisiert bestimmt** werden.
- I Aktuelle Künstliche Neuronale Netze aus dem Bereich des Deep Learning haben gewöhnlich mehrere Millionen Parameter, die **automatisiert angelernt** werden, um beispielsweise Objekte wie Straßenschilder oder Personen in Bilddaten zu erkennen.

Testen in klassischen softwarebasierten Systemen

- I Da oftmals nicht klar ist, wie das System auf **unbekannte Eingabedaten** reagieren wird, lässt sich so auch das zukünftige Verhalten zum Zeitpunkt der Entwicklung und Inbetriebnahme nicht absolut verlässlich vorhersagen.
- I Bewertung nur möglich für die Eingabedaten, mit denen getestet wurde
- I Allerdings **können in der Regel nicht alle potenziellen Eingabedaten** im Rahmen des Entwicklungsprozesses **durchgetestet** werden.

Motivation Teststrategie

- I Konzeption der Teststrategie und des –designs
 - I Identifikation einer Vielzahl an Fragen, die gestellt werden sollten
 - I → bedarfsorientierte Teststrategie
- I Beispiel der Komplexität:
 - I Ziel: eine **allgemeingültige Absicherungsmethodik** für die KI in autonomen Fahrzeugen
 - I Verantwortlichkeit, **kritische Ausnahmesituationen** systematisch zu berücksichtigen
 - I Sensorik (Datenlage und Qualität)
 - I KI-System
 - I reale Umgebung
 - Im übertragenen Sinn: innerhalb des Straßenverkehrs

A3.3.1: Datenqualität von Trainings- und Testdaten



Häufige Methode: „Wenn die Erkennungsrate nicht ausreicht, nehme ich mehr Daten“.

- Was halten Sie von der Strategie?
- Was erreicht man damit (nicht)?

Herausforderungen beim Testen einer KI

- I 2 wesentliche Herausforderungen beim Testen einer KI:
 - I Datenqualität von Trainings- und Testdaten
 - I Corner Cases

Datenqualität von Trainings- und Testdaten

- I Nicht nur die Qualität der Datenbeschaffenheit spielt eine Rolle
 - I Tiefgründige Probleme:
 - I Vorurteile
 - I Überrepräsentation
 - I ...
- Es existieren **Metriken für die Güte (Qualität) eines Modells**
- Es existieren **keine Metriken für die Qualität der Daten**

A3.3.2: Datenqualität von Trainings- und Testdaten



- Welche Metriken können für die Güte (Qualität) eines Modells herangezogen werden?
- Wie könnten Metriken für die Qualität der Daten aussehen/ nach welchen Kriterien sollten die Daten ausgewählt werden?

A3.3.3: Datenqualität von Trainings- und Testdaten



Überlegen Sie sich (ggf. eine Recherche), inwiefern die Datenauswahl oder -verfügbarkeit die Qualität einer KI beeinflussen kann.

Validierung von maschinell gelernten Modellen

- I systematische Identifizierung von potentiellen Ausnahmesituationen
 - I Neben der **Quantität der Daten (Menge)** ist auch die **Qualität der Daten (hier Vielfalt)** entscheidend
 - I Sogenannte **Corner Cases** (auch Edge Cases) bilden unwahrscheinliche Szenarien/Kombinationen ab

- I Teststrategie
 - I sorgfältig ausgearbeitete Testfälle
 - I Testautomatisierung
 - I exploratives Testen

- I erfordert ein sehr **gutes Verständnis der Domäne**

A3.3.4: Corner Cases



- ┃ Recherchieren Sie den Begriff „Corner Cases“.
- ┃ Was steckt dahinter?

A3.3.5: Corner Cases



Kennen Sie (prominente) Beispiele, in denen das Nicht-Berücksichtigen von Corner Cases zu Problemen führte?

A3.3.6: Corner Cases



Welche Corner Cases sind in Ihrem Szenario der Vorbereitungsaufgabe sinnvoll?

A3.3.7: Corner Cases



- Was ist der „schwarze Schwan“ (black swan)?
- Was hat es mit dem Testen der KI zu tun?



A3.3.8: Teststrategie



Entwickeln Sie eine Teststrategie für den Use Case Ihrer Vorbereitungsaufgabe (alternativ: Autonomes Fahren oder sonstiger Use Case).

3.4 Assurance Cases als Lösung zur Absicherung von KI?

Assurance Cases

- I **Herkömmliche Methoden der Qualitätssicherung** sowie existierende Standards sind für den Einsatz von KI **nicht ausreichend**
- I → es kommt darauf an, sehr ausführlich **darzulegen** und zu **begründen**, warum der Einsatz einer KI-Komponente sicher ist
 - I keine inakzeptablen Risiken
- I Eine **Argumentation**, kann helfen, um darzulegen, dass ein System hinreichend verlässlich ist, und daraus zu lernen
 - I sollte den anwendungsfall- und technologiespezifischen Besonderheiten eines Use Cases gerecht werden
- I Hersteller und Prüfer benötigen für diese komplexe Argumentation praktikable Methoden und Ansätze

Assurance Cases

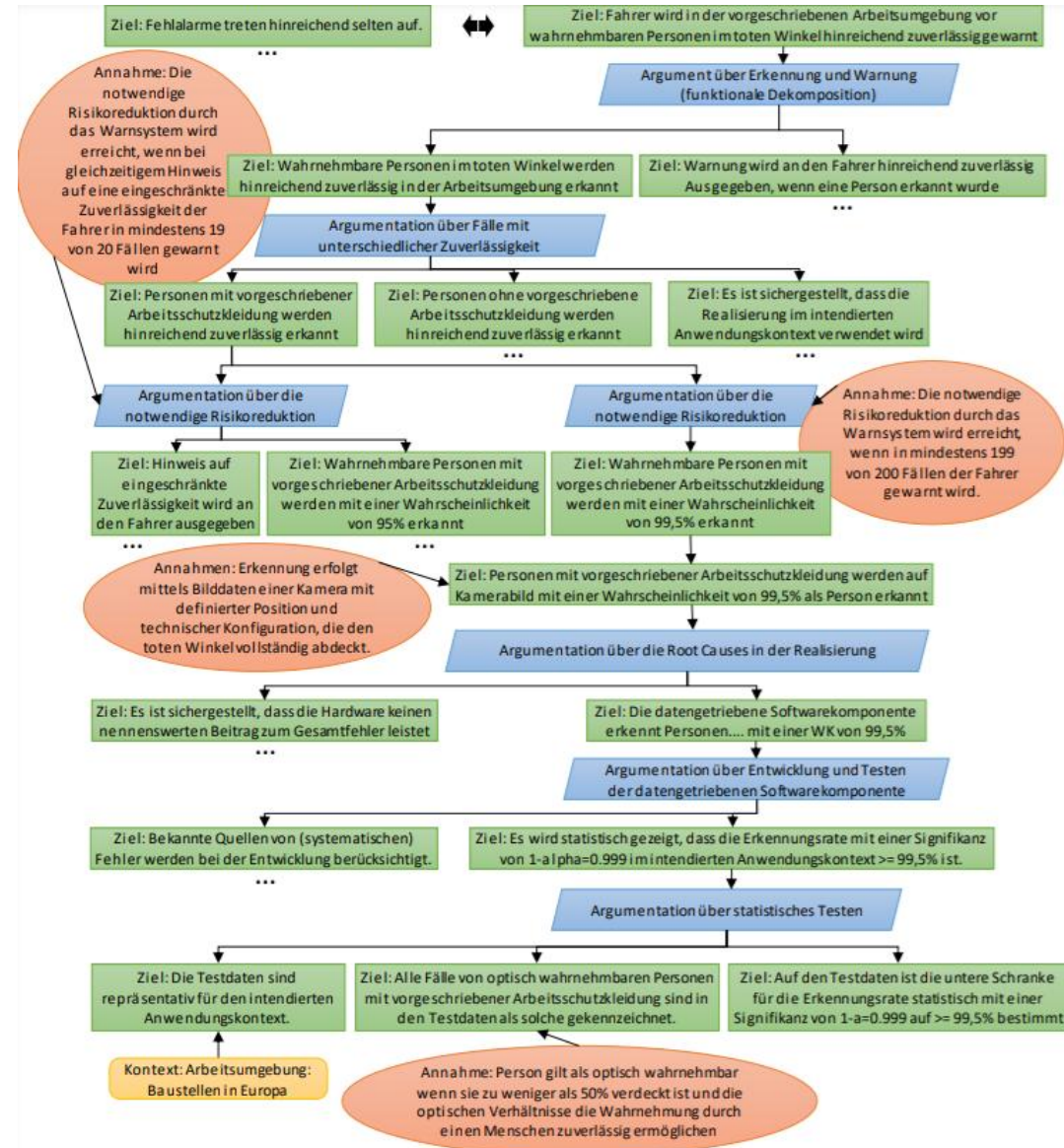
- I Verwendung von **Assurance Cases (AC)**
 - I das Qualitätsziel während der Entwicklung nicht aus den Augen verlieren
 - I nach der Entwicklung argumentativ belegen zu können, warum es erfüllt wurde
 - I komplexe Sicherheitsargumentationen zu dokumentieren
 - I bilden die notwendige umfassende Argumentation ab

- I Assurance Cases erklären stichhaltig, warum ein bestimmtes Ziel erreicht wurde.

- I Assurance Cases sind insbesondere in der Sicherheitstechnik weit verbreitet
 - I Es gewinnt zunehmend an Bedeutung, wenn es darum geht, darzulegen, warum ein System mit kritischen ML-Komponenten sicher ist.

Assurance Cases

Beispiel Assurance Cases
I (wird später konkreter behandelt)



Assurance Cases

Assurance Cases ist eine klar strukturierte, übersichtliche Argumentation, die umfassend begründet, warum ein System bestimmte Anforderungen in einem konkreten Anwendungskontext erfüllt.

Definition Assurance Cases nach ISO/IEC/IEEE 15026-1:2019

„reasoned, auditable artefact created that supports the contention that its top-level claim (or set of claims) is satisfied, including systematic argumentation and its underlying evidence and explicit assumptions that support the claim(s)“.

Warum sind Assurance Cases ein geeignetes Instrument um die Safety trotz KI nachzuweisen?

- I Assurance Cases sind geeignet, wenn es kein „Schema F“ gibt, um Safety nachzuweisen
 - I Kein „Schema F“ für KI in Sicherheitsfunktionen – keine Maßnahmen für KI in SIL-Tabellen
 - I Zielorientiertheit von AC fördert die Auswahl von effektiven und ausreichenden Maßnahmen!
 - I AC sind Kernelement von neuen Normen für Safety von autonomen Systemen mit KI (UL 4600, VDE-AR-E 2842-61)
- I Assurance Cases sind geeignet, um aus Felderfahrungen zu lernen
 - I Bei KI sind Marktüberwachung und Felderfahrung besonders relevant
 - I Safety Performance Indikatoren für Behauptungen und Annahmen schließen Wissenslücken

Warum sind Assurance Cases ein geeignetes Instrument, um die Safety trotz KI nachzuweisen?

- I **Assurance Cases unterstützen modulare Zertifizierung** im Kontext von Industrial Internet of Things (IIoT)
 - I Zertifizierung von KI soll für verschiedene Maschinen und deren Anwendungskontext möglich sein
 - I Ein AC kann die Garantien für eine KI und den betrachteten Kontext modular begründen
 - I Maschinenhersteller können AC von „KI-Herstellern“ in ihren AC integrieren

Assurance Cases

- I Für besonders wichtige Qualitätsziele (Sicherheitsziele) sollte man einen Assurance Case aufstellen und erklären, wie die **Qualitätssicherungsmaßnahmen** zum **Erreichen des Ziels** beitragen und warum sie ausreichen.
- I Beim Testen von ML-Komponenten wirft dies dann folgende fundamentale Frage auf: **"Welche Aussagekraft hat das Testen für mein Qualitätsziel?"**

Assurance Cases

- I Bei "normal" programmierten Komponenten liefert **systematisches Testen** Indizien dafür, dass kein grundlegender Denkfehler beim Aufstellen der **kausalen Zusammenhänge** zwischen Eingabe und Ausgabe gemacht wurde und, dass die Zusammenhänge auch korrekt im Code umgesetzt wurden.
- I Welche Aussagekraft hat aber das Testen, wenn die **Kausalitäten** in einer Spezifikation durch **Korrelationen** zwischen Daten **ersetzt** werden?



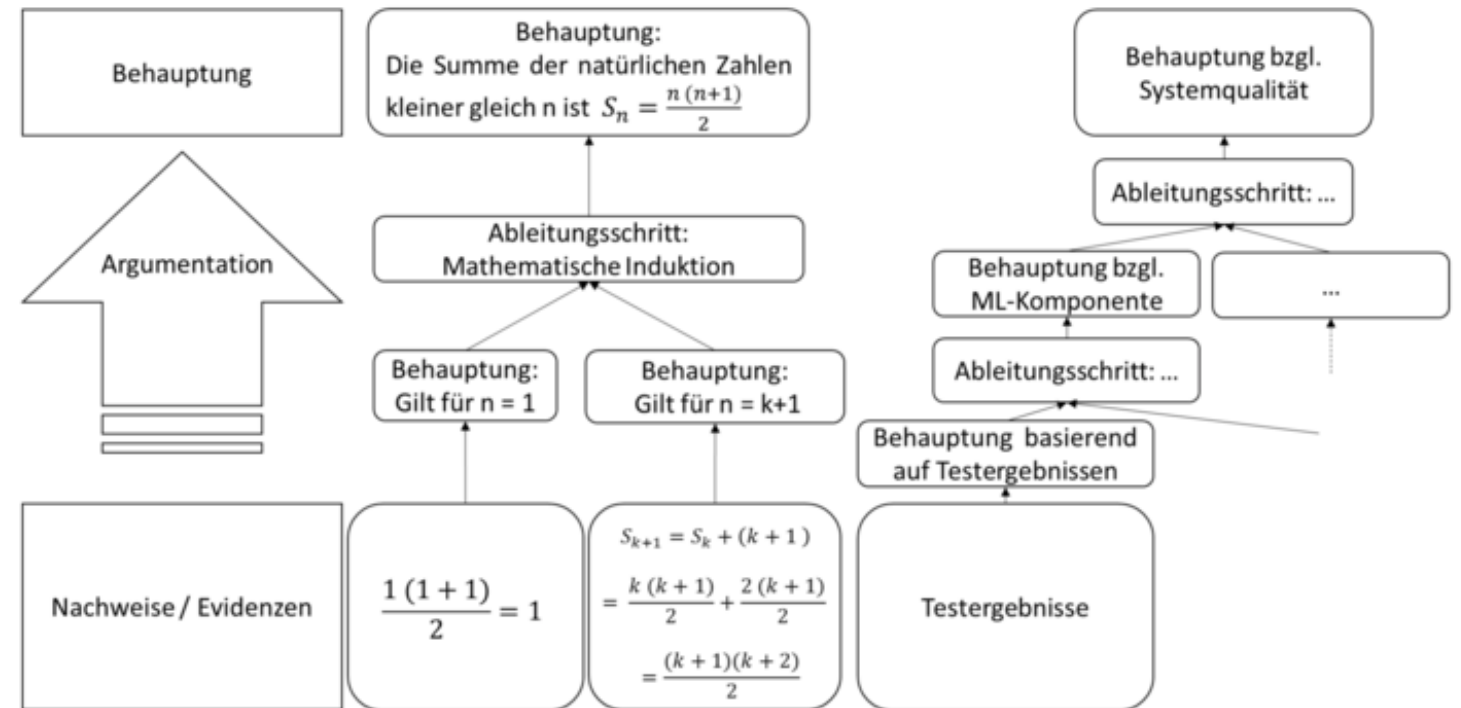
© marketoonist.com

Assurance Cases

- I Assurance Cases erklären in einer auf Nachweisen basierenden Argumentation, **warum eine bestimmte Behauptung** gilt.
- I Die Argumentation bricht die Behauptung auf andere Behauptungen herunter, die dann durch Nachweise belegt werden.
- I Auf der nächsten Folie wird dies anhand eines Induktionsbeweises veranschaulicht.

Assurance Cases

- Die initiale Behauptung wird durch einen Ableitungsschritt in zwei Behauptungen zerlegt.
 - In der Regel hat die Argumentation aber mehrere Ableitungsschritte.
- Wie im rechten Teil der Abbildung dargestellt, ergibt sich dadurch eine baumartige Struktur.



Assurance Cases

- I Die grafische Darstellung macht die Argumentation übersichtlicher und einfacher anzupassen als eine lineare Argumentationskette in einem Textdokument.
- I Die explizite Darstellung jedes einzelnen Ableitungsschritts ermöglicht eine systematische Überprüfung.
- I **top-down**
 - I Prüfung, ob die abgeleiteten Behauptungen vollständig sind
- I **bottom-up**
 - I Prüfung, ob die abgeleitete Behauptung wirklich impliziert wird

A3.4.1: Assurance Cases Beispiel

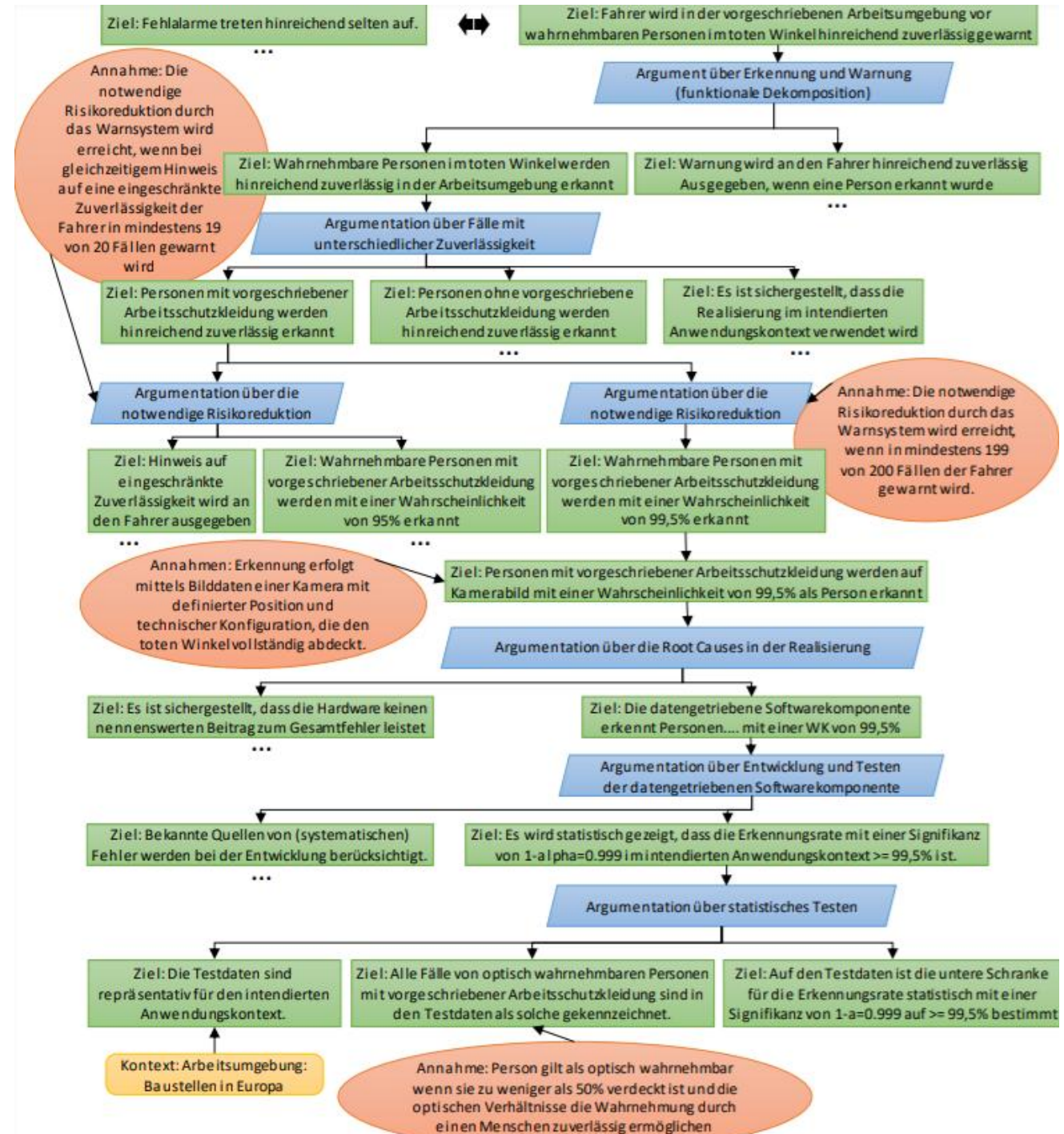


Schauen Sie sich das folgende Beispiel zu Assurance Cases an und machen Sie sich mit dem Beispiel vertraut (es wird im Folgenden noch verwendet).

- I Erklären Sie, wie die KI-Funktion in der Anwendung sichergestellt wird.
- I https://www.informatik-aktuell.de/fileadmin/templates/wr/pics/Artikel/03_Betrieb/KI/Abb2_KI-Testen_Heidrich.pdf

Assurance Cases Beispiel

https://www.informatik-aktuell.de/fileadmin/templates/wr/pics/Artikel/03_Betrieb/KI/Abb2_KI-Testen_Heidrich.pdf



Herausforderungen und Empfehlungen

1. Zielformulierung und Trade-offs

- I Beispiel Qualitätsziel "Der Fahrer wird in der Arbeitsumgebung vor wahrnehmbaren Personen im toten Winkel hinreichend zuverlässig gewarnt".
- I Das "hinreichend zuverlässig" adressiert fehlende Warnungen ("**falsch negativ**"), diese stehen aber in einer Beziehung zu den ebenfalls ungewollten Fehlalarmen ("**falsch positiv**").
- I **Die Fehlerbilder und ihre Minimierung stehen im Konflikt miteinander.**

Ziel: Fehlalarme treten hinreichend selten auf.



Ziel: Fahrer wird in der vorgeschriebenen Arbeitsumgebung vor wahrnehmbaren Personen im toten Winkel hinreichend zuverlässig gewarnt

Herausforderungen und Empfehlungen

1. Zielformulierung und Trade-offs

Empfehlung:

- I Die Argumentation muss erklären, wie dieser **Konflikt aufgelöst** und die initial abstrakten Ziele in konkrete, aber nicht widersprüchliche Anforderungen heruntergebrochen werden.
- I Weiterhin muss sie darlegen, warum die Fehlerwahrscheinlichkeiten "**hinreichend niedrig**" sind (unter Berücksichtigung des realistisch Möglichen)
 - I eine Restwahrscheinlichkeit wird es aufgrund zufälliger Fehler immer geben
- I Ein Qualitätsziel sollte daher auch **keine absoluten Vorgaben** mit Wörtern wie "**immer**" oder "**niemals**" machen, denn diese sind per se in diesem Kontext nicht erfüllbar.

Herausforderungen und Empfehlungen

2. Funktionale Dekomposition bezüglich KI

- I **Ziel schrittweise gemäß den Aufgaben zerlegen**, die für die Zielerreichung notwendig sind
- I Beispielsweise kann das zuvor genannte Ziel in die beiden Ziele zerlegt werden
 - I (1) "Wahrnehmbare Personen im toten Winkel werden hinreichend zuverlässig in der Arbeitsumgebung erkannt" und
 - I (2) "Warnung wird an den Fahrer ausgegeben, wenn eine Person erkannt wurde"
- I „Teile und herrsche“



Herausforderungen und Empfehlungen

2. Funktionale Dekomposition bezüglich KI

- I Ein **Abbruchkriterium** für die weitere funktionale Dekomposition ist dabei eine Ebene, auf der Aufgaben eindeutig konkreten Komponenten zuordnet werden können.
 - I das **linke Ziel lässt sich einer datengetriebenen Komponente** im System zuordnen
 - I das **rechte Ziel wird mit klassischen Hardware- und Softwarekomponenten** realisiert
 - I zu deren Absicherung existieren etablierte Standards, auf die wir daher hier nicht weiter eingehen werden.
- **Empfehlung:** Eine Dekomposition sollte zunächst funktional bis auf die Ebene konkreter Komponenten erfolgen.

Herausforderungen und Empfehlungen

3. Dekomposition bezüglich notwendiger Zuverlässigkeit

- I Datengetriebene Komponenten erfüllen ihre Aufgabe normalerweise **nicht in jeder Situation mit dem gleichen Grad an Zuverlässigkeit**.
 - I Die Zuverlässigkeit einer Objekterkennung hängt beispielsweise unter anderem von den vorliegenden Lichtverhältnissen ab.
 - I Es gibt also bestimmte Situationen, beispielsweise bei starker Dunkelheit, bei denen man sich weniger auf die Korrektheit einer Ausgabe verlassen kann
- I Ebenso gibt es **bestimmte Situationen**, bei denen wir **geringere Anforderungen** an die Zuverlässigkeit haben.
 - I Dies sind Situationen, die weniger riskant sind oder in denen eine höhere Risikoakzeptanz unterstellt werden kann.
 - I Bezogen auf unser Beispiel kann man sich vorstellen, dass alle Personen in der Arbeitsumgebung verpflichtet sind, Schutzkleidung zu tragen, und es daher als akzeptabel angesehen werden kann, wenn Personen ohne Schutzkleidung weniger zuverlässig erkannt werden.

Herausforderungen und Empfehlungen

3. Dekomposition bezüglich notwendiger Zuverlässigkeit

- I Der Assurance Case muss erklären, **warum** die Ausgabe für alle Eingabebereiche **hinreichend zuverlässig** ist.
- I Das legt eine Einteilung der Eingabebereiche und eine entsprechende Fallunterscheidung im Assurance Case nahe.
- I Zunächst sollte dabei bezüglich der geforderten Zuverlässigkeit unterschieden werden.



Herausforderungen und Empfehlungen

3. Dekomposition bezüglich notwendiger Zuverlässigkeit

- I Es gibt verschiedene denkbare Argumente für unterschiedliche **Grade an Zuverlässigkeit**.
- I Das Risiko ist eine **Kombination** aus der Wahrscheinlichkeit (1) der Situation, (2) einer ausbleibenden Warnung in der Situation, (3) dem Risiko, dass es bei ausbleibender Warnung zu einem Unfall kommt und (4) der Schwere eines solchen Unfalls.
- I Die Wahrscheinlichkeit der Situation beeinflusst also das resultierende Risiko.
 - I Entsprechend könnte man der Argumentation folgen, dass das System im Fall fehlender Schutzkleidung nicht so zuverlässig sein muss, da der Fall sehr selten auftritt.
- I Es ist auch denkbar, ein Eigenverschulden einer Person beziehungsweise einen Regelverstoß mit in die Argumentation einfließen zu lassen.
- I Natürlich darf man **nicht unnötig viele Fälle** konstruieren und anschließend argumentieren, dass jeder einzelne Fall hinreichend unwahrscheinlich ist.

Herausforderungen und Empfehlungen

3. Dekomposition bezüglich notwendiger Zuverlässigkeit

Empfehlung:

- I Eine Dekomposition sollte anhand von Zuverlässigkeitsanforderungen erfolgen, die sich aus unterscheidbaren Risikoklassen bzw. Risikoakzeptanzniveaus begründen.

Herausforderungen und Empfehlungen

4. Scope Compliance

- I Die **Fallunterscheidung muss vollständig sein** und es darf kein Fall vergessen werden.
- I Eine lückenlose Zerlegung und realistische Risikoabschätzung sind jedoch nur dann möglich, wenn wir Annahmen über den Anwendungsbereich machen.
 - I So wird in der beispielhaften Zerlegung die Annahme gemacht, dass alle Personen Schutzkleidung tragen müssen.
- I Diese Annahme ist aber nur haltbar, wenn wir die "vorgeschriebene Arbeitsumgebung" klar definieren (z.B. vorschriftsmäßig abgesicherte Baustellen in der EU) und Situationen zuverlässig verhindert oder erkannt werden, in denen das System außerhalb des Anwendungsbereiches operieren würde.
- I So wird in unserem Beispiel im Rahmen eines dritten Ziels explizit gefordert, "dass die Realisierung im intendierten Anwendungskontext verwendet wird."

Herausforderungen und Empfehlungen

4. Scope Compliance

Empfehlung:

- I Der intendierte Anwendungsbereich muss klar definiert und eine "missbräuchliche" Verwendung sollte entsprechend verhindert oder zumindest detektiert werden können.

Herausforderungen und Empfehlungen

5. Quantifizierung der notwendigen Zuverlässigkeit

- I Bei der **Dekomposition** der notwendigen Zuverlässigkeit kann die Fallunterscheidung **auf den höheren Zielebenen noch rein qualitativ** erfolgen, ohne die unterschiedlichen Anforderungen bezüglich der Zuverlässigkeit zu quantifizieren.
- I Eine **Quantifizierung** wird aber spätestens dann erforderlich, wenn man systematisch ableiten möchte, **wie viele Testfälle** benötigt werden und wie viele der Tests erfolgreich sein müssen.

Herausforderungen und Empfehlungen

5. Quantifizierung der notwendigen Zuverlässigkeit

- I Es ist allerdings nicht üblich, quantitative Zuverlässigkeitsanforderungen an Softwarekomponenten zu stellen.
- I Wenn ein **Fehler** beim Black-Box-Testen von **sicherheitskritischer Software** gefunden wird, dann **muss dieser beseitigt** werden.
- I Die Kritikalität gibt daher vor, welche Testmethoden angewendet werden sollen, aber nicht, wie viel Prozent der Testfälle bestanden werden müssen.
- I **Fehlerraten gibt es nur für zufällige Hardwarefehler.**
- I Diese sind aber so gering, dass entsprechende Anforderungen an eine datengetriebene Komponente nicht erfüllbar wären.

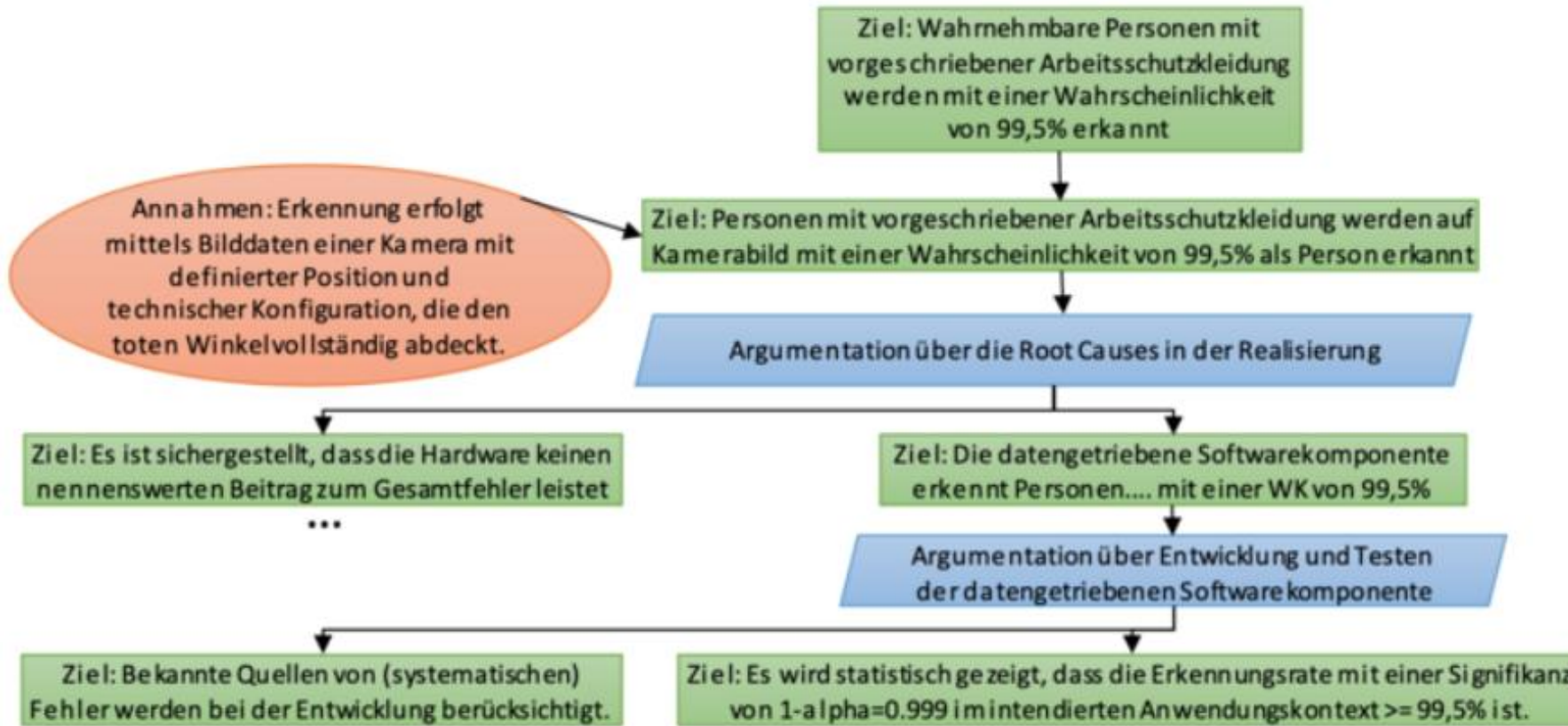
Herausforderungen und Empfehlungen

5. Quantifizierung der notwendigen Zuverlässigkeit

- I Trotzdem lassen sich valide Argumente für den Einsatz von datengetriebenen Komponenten in sicherheitskritischen Bereichen finden.
- I Der Einsatz kann beispielsweise dann begründet werden, wenn die datengetriebene Komponente eine Funktion ermöglicht, die das System insgesamt sicherer macht.
- I So kann beispielsweise die Tatsache, dass ein "Notbremsassistent" auf dem aktuellen Stand der Technik nicht jede Kollision verhindern kann, kein valides Sicherheitsargument dafür sein, keinen "Notbremsassistenten" in Fahrzeugen zu verbauen.

Herausforderungen und Empfehlungen

5. Quantifizierung der notwendigen Zuverlässigkeit



Herausforderungen und Empfehlungen

5. Quantifizierung der notwendigen Zuverlässigkeit

Empfehlung:

- I Bei datengetriebenen Komponenten bietet es sich an (im Gegensatz zu klassischer Software), die Zuverlässigkeit mittels Fehlerraten bzw. Wahrscheinlichkeiten zu quantifizieren.

Herausforderungen und Empfehlungen

6. Fehlerquellen bei der Realisierung datengetriebener Komponenten

- I Nachdem die Ziele für eine datengetriebene Komponente quantifiziert wurden, muss geprüft werden, was dies **für die unterschiedlichen Bestandteile der Komponente** bei ihrer Realisierung bedeutet.
- I Klassischerweise wird hierbei zwischen Software- und Hardwarebestandteilen unterschieden.
- I Bei konventionellen Komponenten wird gewöhnlich davon ausgegangen, dass **Hardware eine abschätzbare Wahrscheinlichkeit für Fehler besitzt**, wohingegen **Software nur systematische Fehler** enthält, die durch die Einhaltung bestimmter Praktiken in der Entwicklung hinreichend reduziert werden müssen.

Herausforderungen und Empfehlungen

6. Fehlerquellen bei der Realisierung datengetriebener Komponenten

- I Für **datengetriebene Komponenten** stellt sich diese Betrachtungsweise jedoch als eher **weniger zielführend** dar.
- I Da sich schon die Anforderungen an die Funktionalität der Komponente nicht vollständig spezifizieren lassen, kann schwerlich ein Nachweis geführt werden, dass die Software der Komponente in allen Situationen wie intendiert funktioniert.
 - I Wie sollte beispielsweise für eine beliebige Kombination von Pixeln spezifiziert werden, ob diese eine Person darstellen oder nicht?
- I Daher lassen sich **sinnvolle Anforderungen an die Korrektheit** einer datengetriebenen Komponente auch **ausschließlich probabilistisch** formulieren und entsprechend nachweisen.
- I Hierbei erreichen Modelle auf dem aktuellen Stand der Technik Fehlerraten, die sich häufig in der Größenordnung von 10^{-2} bis 10^{-4} bewegen.

Herausforderungen und Empfehlungen

6. Fehlerquellen bei der Realisierung datengetriebener Komponenten

- I **Hardwarebasierte Fehler** (z.B. durch einen Bit-Flip im Speicher eines Steuergeräts), sind um Größenordnungen **seltener** und führen noch viel seltener zu einer tatsächlichen Verfälschung der Ausgaben datengetriebener Komponenten.
- I **Gesamtfehlerrate** einer Komponente ergibt sich vereinfacht aus der **Addition** der **hardwarebasierten** und **softwarebasierten** Fehler
- I Daher können die Auswirkungen **hardwarebasierter Fehler** im Vergleich zu den Fehlerraten der eingesetzten datengetriebenen Modelle **häufig vernachlässigt** werden.

Herausforderungen und Empfehlungen

6. Fehlerquellen bei der Realisierung datengetriebener Komponenten

Empfehlung:

- I Bei der Abschätzung müssen sowohl software- als auch hardwareverursachte Fehlverhalten berücksichtigt werden, wobei letztere – Stand heute – größenordnungsmäßig jedoch meist vernachlässigbar sind.

Herausforderungen und Empfehlungen

7. Entwicklung und Testen von Software für datengetriebene Komponenten

- I Die **Aktivitäten** bei der softwareseitigen Realisierung einer datengetriebenen Komponente können grob in **Entwicklung und Qualitätssicherung** eingeteilt werden.

Entwicklung

- I die Datensichtung und -aufbereitung
- I die Modellbildung mit Auswahl passender Lernverfahren, Modellstrukturen und Hyperparameter sowie dem automatisierten Erlernen der Modellparameter.
- I Aktuelle Vorgehensweisen sind hierbei meist hochgradig iterativ, geprägt von Intuition, Experimenten sowie hochspezialisiertem Erfahrungswissen.

Qualitätssicherung

→ Im Vergleich zur klassischen Softwareentwicklung bestehen bis dato **keine etablierten Prozesse und Praktiken**, die nachweislich mit hoher Sicherheit zu zuverlässigen Realisierungen führen.

Herausforderungen und Empfehlungen

7. Entwicklung und Testen von Software für datengetriebene Komponenten

- I Obwohl inzwischen intensiv an Qualitätssicherungstechniken im ML-Bereich geforscht wird, ist die **Menge etablierter Ansätze überschaubar**.
- I Viele im Bereich der Softwareentwicklung eingesetzten Techniken lassen sich bisher **nicht einfach auf die Entwicklung datenbasierter Modelle übertragen**.
 - I Beispielsweise erscheinen klassische Codereviews oder statische Codeanalysen wenig hilfreich.
- I Auch grundlegende Ansätze im Bereich klassischer Softwaretests wie Äquivalenzkassenbildung oder die Ausrichtung der Testfälle anhand bestimmter Codeüberdeckungskriterien sind entweder nicht praktikabel oder bezüglich ihres Nutzens äußerst fraglich.

Herausforderungen und Empfehlungen

7. Entwicklung und Testen von Software für datengetriebene Komponenten

- I Als **anerkanntes Qualitätssicherungsverfahren** kann hingegen das **statistische Testen** betrachtet werden, das insbesondere verwendet werden kann, um Ziele mit Evidenzen zu belegen.
- I Beispiel: "es wird statistisch gezeigt, dass die Erkennungsrate mit einer Signifikanz von 0,999 im intendierten Anwendungskontext größer oder gleich 99,5 Prozent ist"

Herausforderungen und Empfehlungen

7. Entwicklung und Testen von Software für datengetriebene Komponenten

Empfehlung:

- I Eine Argumentation, die sich ausschließlich auf die Anwendung etablierter Entwicklungsprozesse und Methoden beruft, erscheint für datengetriebene Komponenten bisher wenig zielführend und sollte vermieden werden.
- I Eine inhaltliche Argumentation, die sich auf Ergebnisse des statistischen Testens stützt, sollte das Rückgrat des Assurance Cases bilden.

Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

Die Argumentation bezüglich der softwareseitigen Zuverlässigkeit einer datengetriebenen Komponente kann sich beispielsweise auf drei Säulen verteilen.

- I Repräsentative Auswahl von Fällen**
- I Korrekte Testfälle (ground truth)**
- I Interpretation der Testergebnisse (Signifikanz)**

Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

In der **ersten Säule** ist sicherzustellen, dass die beim Testen **verwendeten Daten** eine **repräsentative Auswahl von Fällen** aus dem intendierten Anwendungskontext darstellen.

- I Dies hat insbesondere wichtige Implikationen für die **Datenerhebung**.
 - I In unserem Beispiel sollten Testbilder beispielsweise mit einer **vergleichbaren** Kamera an der **intendierten Position** auf **verschiedenen zufällig ausgewählten** europäischen Baustellen gesammelt werden.
 - I Daten von ausschließlich einer Baustelle oder Daten aus den USA wären hingegen offensichtlich nicht repräsentativ und die Testergebnisse damit nicht aussagekräftig.

Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

- I Ein eklatanter **Fehler** wäre auch, die **Testdaten** schon **während der Modellerstellung zum Lernen** der Modellparameter **einzusetzen**, denn auch dann hätten Testergebnisse auf Basis dieser Daten keine statische Aussagekraft mehr.
- I Die notwendige Breite des Testdatensatzes hängt insbesondere von der natürlichen Varianz im Anwendungsumfeld und der relativen Häufigkeit relevanter Ereignisse ab.
- I Hierbei kann es durchaus sinnvoll sein, seltene, aber risikoreiche Fälle überproportional zu ihrer tatsächlichen Häufigkeit im Testdatensatz zu berücksichtigen und die Testergebnisse anschließend mittels Gewichtung auf eine repräsentative Verteilung zurückzuführen.

Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

- I Die **zweite Säule** der Argumentation beim statistischen Testen beruht auf der Annahme, dass die **Testfälle korrekt** sind, d.h. dass die Grundwahrheit (engl. ground truth) zuverlässig bestimmt wurde.
 - I In unserem Beispiel ergibt sich so das Ziel: "Alle Fälle von optisch wahrnehmbaren Personen mit vorgeschriebener Arbeitsschutzkleidung sind in den Testdaten als solche gekennzeichnet."
- I Auch hierbei ergeben sich potenziell semantische Unklarheiten und mögliche Fehlerquellen.
 - I So ist zu klären, wann eine Person als optisch wahrnehmbar gilt, wie sichergestellt wird, dass optisch wahrnehmbare Personen mit hinreichend hoher Wahrscheinlichkeit auf den Bilddaten im Testdatensatz auch vermerkt sind, und wie dies kosteneffizient erfolgen kann.
- I Derzeit ist die Erstellung qualitativ hochwertiger Testdaten noch immer eine **mit hohem manuellem Aufwand und hohen Kosten** verbundene Tätigkeit.

Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

- I Die **dritte Säule** der Argumentation beim statistischen Testen bildet die **Interpretation der Testergebnisse**, wobei zu berücksichtigen ist, dass der Testdatensatz nur einen Auszug (engl. sample) aus dem intendierten Anwendungskontext darstellt.
- I Daher kann eine Aussage zur Zuverlässigkeit im Anwendungskontext auf Basis der Testergebnisse nur bei Akzeptanz einer **Irrtumswahrscheinlichkeit** Alpha erfolgen.
- I Diese bestimmt die Signifikanz der Aussage, die auf Basis der Testergebnisse bezüglich der Zuverlässigkeit getroffen wird.

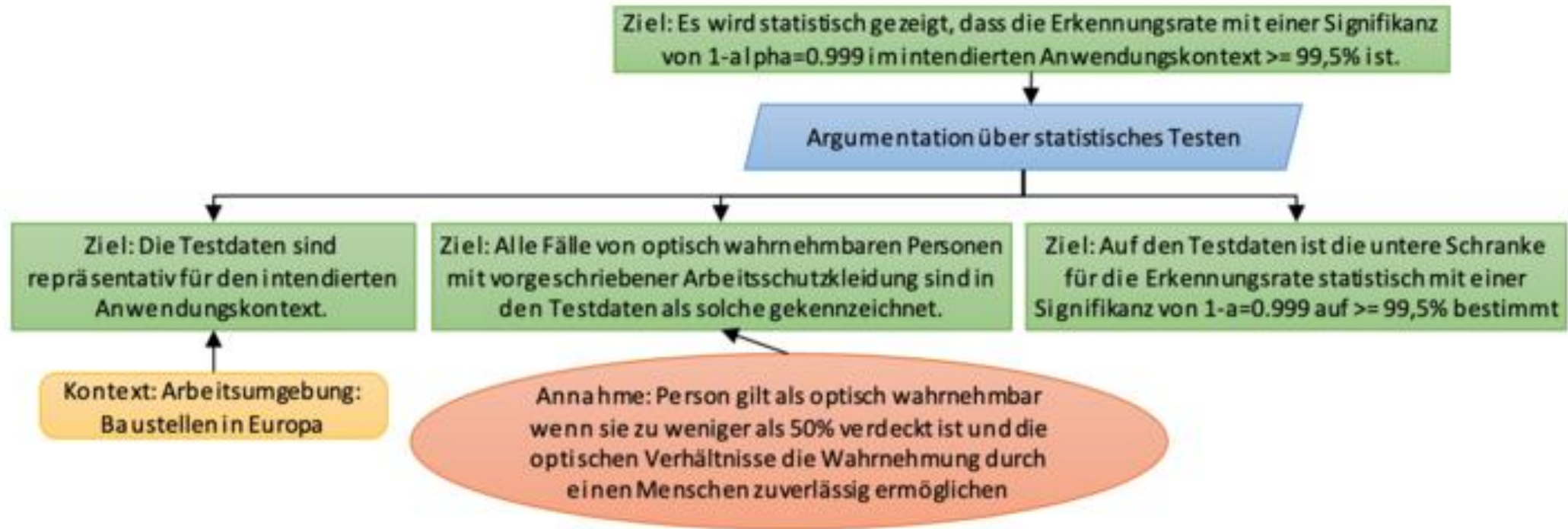
Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

- I **Ohne** die Angabe einer **Irrtumswahrscheinlichkeit** bzw. Signifikanz des Tests sind hingegen Informationen hinsichtlich der Zuverlässigkeit/Erkennungsrate **nicht valide zu interpretieren**.
- I Dies mag das folgende Extrembeispiel illustrieren: Die Komponente wird auf einem Testdatensatz mit genau einem zufälligen Testfall aus dem Anwendungskontext geprüft und liefert für diesen ein korrektes Ergebnis.
- I Daraus jedoch zu folgern, dass die Komponente im Anwendungskontext eine hundertprozentige Zuverlässigkeit aufweist, wäre offensichtlich fatal.

Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen



Herausforderungen und Empfehlungen

8. Zuverlässigkeitsargumentation über statistisches Testen

Empfehlungen:

- I Repräsentativität der verwendeten Testdaten muss sichergestellt werden können.
- I Erstellung eines qualitativ hochwertigen, insbesondere korrekt annotierten, Testdatensatzes wird empfohlen – auch wenn dies aufwändig sein kann.
- I Auswertung und Interpretation der Testergebnisse muss immer unter Berücksichtigung in der Argumentation der geforderten Signifikanz erfolgen.

A3.4.2: Assurance Cases



- I Nehmen Sie nochmal ihr Szenario aus der Vorbereitungsaufgabe zur Hand.
- I Entwickeln Sie ein Assurance Cases für Ihr Szenario.
- I Sollte Ihr Szenario unter den Dimensionen „nicht viel hergeben“, wählen Sie sich ein anderes Szenario.
 - I Weitere Beispiele:
 - I KI für Autonomes Fahren (Spurerkennung o.ä.)
 - I KI für Qualitätsüberwachung der Produkte einer Produktion

Zusammenfassung

Zusammenfassung

- I Herkömmliche Methoden der Qualitätssicherung sowie existierende Standards sind für den Einsatz von KI nicht ausreichend
- I Unsicherheiten beim Einsatz von ML/KI
 - I Erkennungswahrscheinlichkeit und Konfidenz
- I Herausforderungen beim Testen einer KI
 - I Datenqualität
 - I Corner Cases
- I Strategien und Frameworks zum systematischen Testen
 - I Verwendung von Assurance Cases zur Erklärung, warum ein bestimmtes Ziel erreicht wurde

Literaturnachweise

- [Kläs 2021] M. Kläs, A. Jedlitschka: Maschinelles Lernen und KI – warum tun wir uns im Engineering schwer damit?
- [Kläs 2018] Michael Kläs and Anna Maria Vollmer, "Uncertainty in Machine Learning Applications – A Practice-Driven Classification of Uncertainty", First International Workshop on Artificial Intelligence Safety Engineering Sept 18th, 2018, Västerås, Sweden.
- [Kläs 2021a] Michael Kläs, Rasmus Adler, Jens Heidrich: Testen im Zeitalter von KI;
<https://www.informatik-aktuell.de/betrieb/kuenstliche-intelligenz/testen-im-zeitalter-von-ki.html>
- [Beining 2021] Leonie Beining: KI in der Industrie absichern & prüfen - Was leisten Assurance Cases
[ki_in_der_industrie_sichern_und_pruefen.pdf \(stiftung-nv.de\)](#)
- [Adler 2021] Rasmus Adler: Assurance Cases als Prüf- und Zertifizierungsgrundlage von KI [Safety Supervisor \(gi.de\)](#)



Fragen?